

Course Project Report

Course: CY321 ? Secure Software Design with Secure Coding Implementation

Project Title: AUI Secure ? DevSecOps CI/CD Pipeline & Secure Coding Demonstration Platform

Team: 3-person team

Submission Date: 2026-05-07

1. Executive Summary

AUI Secure is a Flask-based web application that demonstrates every OWASP Top-10 vulnerability in two forms simultaneously: a deliberately vulnerable endpoint for education and a hardened equivalent showing the correct defence. A server-side feature flag (SECURE_MODE) keeps all vulnerable endpoints inert by default, making the same codebase both a production-safe application and an interactive teaching platform.

Wrapped around the application is a 5-stage DevSecOps CI/CD pipeline that gates every push to main with automated security checks: functional tests, secret scanning, static code analysis (SAST), dependency vulnerability scanning (SCA), and dynamic application security testing (DAST).

Key Results:

- 62/62 automated pytest cases pass (functional + security)
- 0 Bandit High-severity High-confidence issues across 1,402 lines of code
- 0 known dependency CVEs (pip-audit ? 57 packages checked)
- 0 secrets in repository history (gitleaks, full history scan)
- All 7 HTTP security headers correctly emitted
- 6 security findings discovered during final review; all fixed
- Full OWASP Top-10 coverage (9 demonstrated + mitigated; A10 SSRF not applicable by design)

2. Project Description

2.1 Motivation and Goals

The goal was to build a project that demonstrates secure software development *in practice*, not just in theory. Instead of a generic web app with some security features bolted on, we built a dual-layer platform:

- Layer 1: A fully secure web application implementing real-world security patterns
- Layer 2: An automated security pipeline that verifies those patterns on every code change

The platform serves as both the submission artefact and a teaching tool for the course.

2.2 Technology Stack

Component	Technology	Version
Web framework	Flask	3.1.3
WSGI server	Werkzeug (dev) / Gunicorn (prod)	3.1.8
Database	SQLite (dev) / PostgreSQL (prod)	?
Password hashing	werkzeug crypt	Memory-hard
Rate limiting	Flask-Limiter	4.1.1
HTML sanitisation	bleach	6.3.0

Testing	pytest	9.0.3
SAST	Bandit	1.9.4
SCA	pip-audit	2.10.0
Secret scanning	gitleaks	v8.18
DAST	OWASP ZAP	Baseline
CI/CD	GitHub Actions	?
Pre-commit	pre-commit	4.6.0

2.3 Project Architecture

```

    Browser / API Client
    ?
    HTTPS (prod)
    ?
    ?????????????????????????????????????????????????????????
    ?      Flask Application      ?
    ?                               ?
    ? before_request: generate CSP nonce      ?
    ? after_request : 7 security headers + CSP ?
    ?                               ?
    ? ?????????? ?????????? ??????????      ?
    ? ? auth ? ? items ? ?api_keys? ...      ?
    ? ?????????? ?????????? ??????????      ?
    ?      ?????????????????????????????      ?
    ? @requires_auth / @requires_role      ?
    ? csrf_protect / validators      ?
    ?      ?                               ?
    ? ?????????????????????????????????      ?
    ? ? audit.log_action ? DB      ?
    ? ?????????????????????????????????      ?
    ?????????????????????????????????????????????????????????
    ? parameterised SQL only
    SQLite DB
    (users, items, api_keys,
    audit_logs, account_lockouts,
    demo_products)

    CI/CD Gate (every push):
    pytest ? gitleaks ? bandit ? pip-audit (+ remediate.py) ? ZAP
  
```

3. Deliverable Mapping

Deliverable 1 ? Security Planning

Folder: Deliverable-1/

Submission date: Earlier in semester

Documents produced:

- Project_Proposal.pdf ? Project concept and scope
- D1_Security_Requirements_Planning.pdf ? Security requirements mapped to OWASP Top 10
- D1_Risk_Management.pdf ? Risk register and risk matrix

- D1_Threat_Modeling.pdf ? STRIDE threat model

How our code maps to D1:

D1 Requirement	Code Implementation	File
Secure authentication	script hashing, session rotation, logout, CSRF	auth.py, security.py
Authorisation / RBAC	@requires_auth, @requires_role("admin")	rbac.py
Input validation	Username regex, password policy, HTML stripping, SQLi blocklist	validators.py
SQL injection prevention	Parameterised ? queries everywhere on secure	items.py, auth.py, db.py
XSS prevention	bleach.clean() + html.escape() + CSP nonce	validators.py, security.py
Security headers	7 headers including CSP, X-Frame-Options,	security.py
Rate limiting	Flask-Limiter per-route caps	security.py, auth.py
Audit logging	Every action logged with user, IP, UA, timestamp	audit.py
Threat model controls	All STRIDE threats have corresponding code controls	See Section 6

Deliverable 2 ? Initial Implementation

Folder: Deliverable-2/

Submission date: 19/04/2026

Contents: Snapshot of the working Flask application at the time of submission.

What was implemented by D2:

- Full application factory pattern (main.py)
- Authentication system with logout (auth.py)
- RBAC decorators (rbac.py)
- Security headers and CSRF (security.py)
- Input validation and sanitisation (validators.py)
- Secure and insecure search endpoints (items.py)
- API key management (api_keys.py)
- Admin dashboard (dashboard.py)
- Interactive lab endpoints (lab.py)
- IDOR demo (profiles.py)
- Full test suite (62 tests)
- 5-stage CI/CD pipeline (.github/workflows/devsecops.yml)
- Pre-commit hooks (.pre-commit-config.yaml)

Issues present in D2 (fixed in D3):

- F1: IDOR attack endpoint not gated by SECURE_MODE
- F2: time.sleep() in login handler (DoS amplifier)
- F3: SECURE_MODE toggle accessible to any user
- F4: Import placed mid-file
- F5: No admin demo account
- F6: CSP nonce broke inline onclick handlers

Deliverable 3 ? Final Implementation & Security Testing

Folder: Deliverable-3/

Submission date: 2026-05-07

Contents: Hardened source + full security analysis.

Documents produced:

- Final_Report.md ? Project overview and OWASP mapping
- Secure_Code_Review.md ? Line-by-line code review of all 14 modules
- Security_Testing_Analysis.md ? SAST + SCA + DAST + pytest results
- Security_Fixes_Changelog.md ? Before/after diffs for all 6 fixes

All 6 findings fixed and re-verified:

- pytest tests/ -v ? 62/62 pass
- bandit -r app/ -ll -ii ? 0 High-severity High-confidence issues
- pip-audit -r requirements.txt ? No known vulnerabilities

4. Security Features Implemented

4.1 Authentication & Session Management

File: auth.py, main.py, security.py

Feature	Mechanism
Password storage	werkzeug.security.generate_password_hash using bcrypt (memory-hard, GPU-resistant) with random salt
Password verification	check_password_hash ? constant-time comparison at the OpenSSL layer
Account lockout	Exponential backoff: base_duration × 2 ^(lockouts) , capped at 86,400 s (24 h)
Timing attack resistance	Dummy hash check for unknown usernames equalises response time
Session rotation	session.clear() runs before writing new session data on every login
Session expiry	Server-side validation of login_time stored in session; checked on every request
CSRF protection	256-bit token (secrets.token_hex(32)), secrets.compare_digest, readable from header/JSON/form
Cookie security	HttpOnly, SameSite=Lax, Secure (production only)

4.2 Authorisation (RBAC)

File: rbac.py

```
@requires_auth      # Requires valid session OR valid API Bearer token
@requires_role("admin") # Requires role='admin' in session (server-side only)
```

- Role is stored in the server-side session, never accepted from the request body
- requires_role logs ACCESS_DENIED events to the audit trail
- API-key authentication (Bearer sk_?) is supported as an alternative to session cookies

4.3 Input Validation

File: validators.py

Validator	Rule
validate_username	3?32 chars, [a-zA-Z0-9_] only ? blocks spaces, @, ../ path traversal
validate_password	?8 chars, must contain uppercase + lowercase + digit + special character

sanitize_text	bleach.clean(tags=[], strip=True) + null-byte removal + length cap
validate_search_query	Blocklist of 7 SQLi patterns ('--', ; drop, union select, etc.)
validate_item_title	HTML-stripped, max 200 chars, required
validate_item_body	HTML-stripped, max 10,000 chars

4.4 SQL Injection Prevention

Files: items.py, auth.py, db.py, api_keys.py, dashboard.py

All production SQL uses parameterised queries:

```
# Correct parameterised form
db.execute("SELECT * FROM items WHERE user_id = ? AND title LIKE ?", (_uid(), f"%{q}%"))

# Intentionally vulnerable (gated by SECURE_MODE=false for demo)
sql = f"SELECT ... WHERE name LIKE '%{q}%'" # NEVER executed in production
```

4.5 XSS Prevention

Files: validators.py, security.py, lab.py

Three-layer defence:

bleach.clean(tags=[], strip=True) ? removes all HTML/XML tags

html.escape() ? converts <, >, ", & to safe HTML entities

CSP script-src 'self' 'nonce-{nonce}' (production) ? even if HTML is rendered, only nonce'd scripts execute

4.6 Security Headers

File: security.py ? applied to every response via @app.after_request

Header	Value	Purpose
Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline' (dev) / 'self' 'nonce-{n}' (prod)	Prevents XSS and data injection
X-Frame-Options	DENY	Prevents clickjacking
X-Content-Type-Options	nosniff	Prevents MIME confusion attacks
X-XSS-Protection	1; mode=block	Legacy XSS auditor (defence in depth)
Referrer-Policy	strict-origin-when-cross-origin	Prevents URL-based info leakage
Permissions-Policy	geolocation=(), camera=(), microphone=()	Disables sensor APIs
Strict-Transport-Security	max-age=31536000; includeSubDomains	Forces HTTPS (production only)

4.7 Rate Limiting

File: security.py, auth.py

- Default limits: 200 requests/day, 50/hour per IP
- Auth endpoints (/auth/register, /auth/login): 10/minute
- Exceeding the limit returns 429 Too Many Requests

4.8 API Key Security

File: api_keys.py

- Keys generated with "sk_" + secrets.token_urlsafe(32) ? 256 bits of entropy
- Stored as SHA-256 hash ? the raw key is never stored
- Only the first 10 characters (prefix) are stored for UI display
- last_used timestamp updated on every authenticated call
- Soft revocation: is_active=0 (preserves audit history)

4.9 Audit Logging

File: audit.py

Every state-changing action is logged:

- user_id, username, IP address, User-Agent
- action (e.g., LOGIN_SUCCESS, LAB_SQLI_ATTACK, ACCESS_DENIED)
- resource, details (JSON), timestamp, success flag
- Append-only ? no UPDATE or DELETE on audit_logs table
- Silent failure ? audit errors never propagate to the user

5. CI/CD Security Pipeline

File: .github/workflows/devsecops.yml

Every push to main or PR triggers 5 sequential jobs. A failure at any stage fails the build and blocks the merge.

Stage 1: build-and-test	? pytest 62 tests (SECURE_MODE=true)
Stage 2: secret-scan	? gitleaks v8.18 (full history, fetch-depth: 0)
Stage 3: sast	? bandit -r app/ -ll -ii (High+High gate)
Stage 4: sca	? pip-audit ? remediate.py ? auto-PR if CVE found
Stage 5: dast	? OWASP ZAP baseline vs live Flask app

Auto-remediation (Stage 4): When pip-audit finds a CVE, remediate.py:

- Parses the JSON output
- Bumps the minimum version in requirements.txt
- Pushes a security/auto-fix-deps-<timestamp> branch
- Opens a GitHub PR labelled security for human review

Pre-commit (local gate): black, bandit, pip-audit, gitleaks, YAML/JSON validation, private-key detector, large-file blocker

6. OWASP Top-10 Coverage

#	Category	Demonstrated by	Mitigated by	Tested by
A01 Broken Access Control	GET /users/attack/<id> (gated), GET /dashboard without admin role	RBAC decorators, session-bound _uid(), WHERE user_id = ?		test_non_admin_dashboard_ blocked,
A02 Cryptographic Failures	POST /lab/hash/demo ? shows two different hashes from same input	script + random salt; HttpOnly + SameSite cookies; HSTS in prod		test_password_policy[*], manual cookie inspection

CY321 - Secure Software Design | AUI Secure Project

A03 Injection	POST /lab/sqli/attack, POST /lab/xss/attack (both gated)	Parameterised queries, bleach, html.escape, CSP	test_secure_search_resists_sqli_injection[*] (6 payloads), test_xss_payloads_stripped_f
A04 Insecure Design	The SECURE_MODE toggle pattern itself	Server-side feature flag; SECURE_MODE=true by	test_insecure_search_blocke
A05 Security Misconfiguration	n/a ? we ship secure defaults	7 security headers; CSP; secure cookie; dev/prod split	test_security_headers_prese
A06 Vulnerable & Outdated Components	n/a	pip-audit + auto-remediation; pinned floor versions	CI pipeline (Stage 4)
A07 Identification & Auth Failures	Brute force lab (account lockout demo)	Lockout w/ exponential backoff; session rotation; CSRF; rate limit; F2 fix	test_account_lockout_after_max_attempts,
A08 Software & Data Integrity Failures	n/a	gitleaks prevents committed secrets; signed session	CI pipeline (Stage 2) out
A09 Security Logging & Monitoring	Live audit event feed in dashboard	audit.log_action() on every action; admin dashboard	Dashboard reads audit_logs directly
A10 SSRF	n/a ? demonstrated absent	App makes zero outbound HTTP calls	n/a (attack surface zero)

Score: 10/10 addressed (9 demonstrated + mitigated, 1 not applicable by design)

7. Test Results Summary

Test Gate	Command	Result
Functional tests	pytest tests/test_app.py -v	22/22 PASS
Security tests	pytest tests/test_security.py -v	40/40 PASS
All tests	pytest tests/ -v	62/62 PASS
SAST	bandit -r app/ -ll -ii	0 High-severity High-confidence
SCA	pip-audit -r requirements.txt	No known vulnerabilities
Secret scan	gitleaks detect --source .	No secrets detected
DAST	OWASP ZAP baseline	All security headers present
Manual pentest	All Attack/Defend pairs	All exploits work in SECURE_MODE=false; all blocked in true

8. Security Findings and Fixes

During the Deliverable-3 review phase, 6 security issues were identified and resolved:

ID	Severity	File	Issue	Fix Applied
F1	High	profiles.py	IDOR attack endpoint not gated by SECURE_MODE ? would expose all user profiles (including admin SQLi dump)	

in production

Added SECURE_MODE
gate returning 403

F2	Medium	auth.py	time.sleep() in failed login path blocked worker threads ? DoS amplifier	Removed sleep; added timing-resistant dummy hash for unknown users
F3	Medium	lab.py	SECURE_MODE toggle accessible by any user ? one-step compromise to expose all attack endpoints	Changed from @requires_auth to @requires_role("admin")
F4	Low	items.py	Import statement placed after a function definition	Moved to module top
F5	Low	db.py	No admin demo account ? admin code paths untestable end-to-end	Seeded admin / Admin@1234 (role=admin)
F6	Low	security.py	CSP nonce broke inline onclick= handlers (50+ buttons non-functional)	Split into prod (nonce-only) and dev (unsafe-inline) modes

After fixes: 62/62 tests pass, 0 Bandit issues, 0 pip-audit findings.

9. Lessons Learned

Security must be in the design, not bolted on. The SECURE_MODE toggle is a design-level decision ? it means the application is safe by default and needs explicit action to expose vulnerabilities. This was designed from the start, not added later.

Automated gates catch real bugs. The D3 review process (code reading + automated scanning + behavioural tests) caught 6 real issues including a High-severity production security bug (F1). Without the review, that bug would have shipped.

time.sleep() is the wrong security primitive. F2 shows that developer intuition (sleep to slow down brute force) can create a worse problem (worker exhaustion DoS) than the one it solves. The lockout mechanism is the right control.

CSP is non-trivial to get right. F6 showed that adding a nonce to CSP silently breaks all inline event handlers ? a gotcha that affects many real-world applications. The correct fix depends on whether you're in dev or production.

Pre-commit hooks close the inner loop. Running bandit, pip-audit, and gitleaks locally before every commit means issues are caught in 30 seconds, not 5 minutes into a CI run.

10. Production Deployment Checklist

Item	Status	Action Required
SECRET_KEY env var	Not set in demo	python -c "import secrets; print(secrets.token_urlsafe(64))"
FLASK_ENV=production	Not set in demo	Enables SESSION_COOKIE_SECURE + HSTS
SECURE_MODE=true	Default (correct)	No action needed
WSGI server	Dev server only	Switch to gunicorn or uWSGI

PostgreSQL	SQLite only	Update DATABASE_PATH to SQLAlchemy connection string
Redis rate limiter	Memory backend	Set RATELIMIT_STORAGE_URI=redis://...
HTTPS/TLS	Not configured	nginx/Caddy/cloud load balancer
Change demo passwords	Demo passwords in DB	Drop seeded users or rotate all passwords
Branch protection rules	Not enforced	Require all 5 CI jobs to pass before merge

11. Conclusion

AUI Secure demonstrates that a small team can build a security-first application covering every OWASP Top-10 category through disciplined design, consistent input validation, automated testing, and a shift-left CI/CD pipeline. The five security issues found during the review process were all caught by the same process the project advocates ? confirming that the review methodology itself works.

The final state (62/62 tests, 0 SAST issues, 0 CVEs, 0 secrets) represents a clean security baseline that can be extended, deployed, and used as a teaching reference.